



Learning Objectives

01

Explain the physical, logical, and view levels, schemas and instances, and physical data independence

02

Describe how data models represent information and how relational systems use DDL, DML, and SQL

03

Recognize redundancy and update anomalies and explain how separating relations improves a design

04

Explain the roles of storage, query processing and optimization, and transaction management



Lecture Outline

FOUNDATIONS

01

Data abstraction
and views

02

Schemas and
instances

03

Data models and
database languages

SYSTEM DESIGN

04

Relational
database design

05

DBMS engine
components



Data Abstraction



Levels of Abstraction

Physical

How a record is stored: its physical representation and organization.

Logical

What data are stored and how the data are related.

```
type instructor = record
  ID : string;
  name : string;
  dept_name : string;
  salary : integer;
end;
```

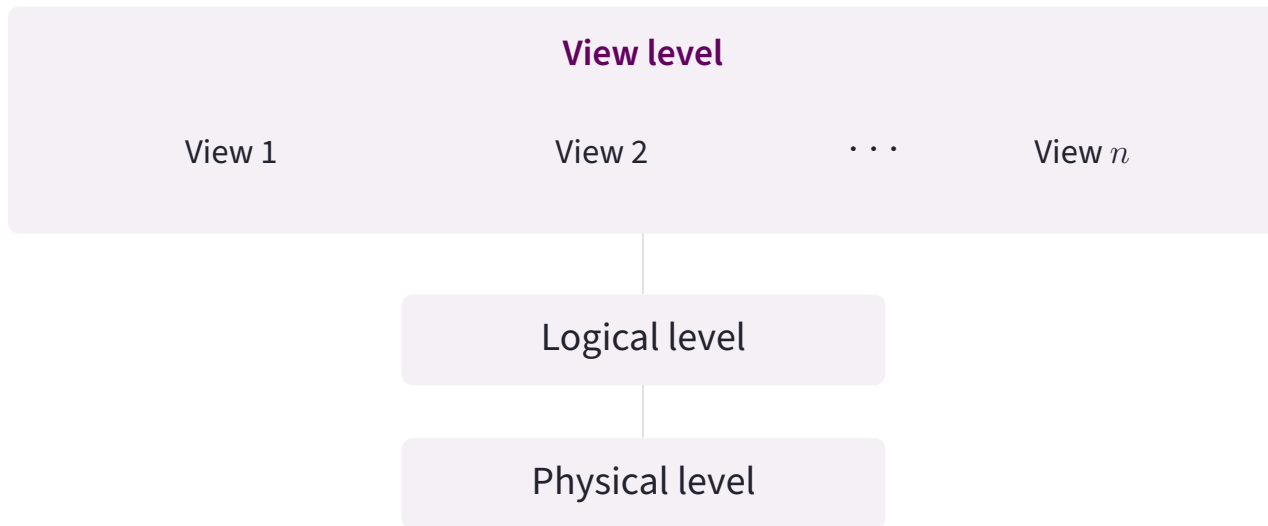
View

Applications **expose** selected information and can **hide** sensitive details, such as salary.



View of Data

An architecture for a database system





Schemas and Instances

A schema is analogous to a variable's type; an instance is analogous to its value.

Logical schema The overall logical structure of the database.

Customer
schema

Name	Customer ID	Account #	National ID	Mobile #
------	-------------	-----------	-------------	----------

Account
schema

Account #	Account type	Interest rate	Min. balance	Balance
-----------	--------------	---------------	--------------	---------

Physical schema The overall physical structure of the database.



Schemas and Instances: The Data

An **instance** is the actual database content at a particular point in time.

CUSTOMER INSTANCE

Name	Customer ID	Account #	National ID	Mobile #
Alice Chen	6728	917322	NID-100001	5550101
Brian Lee	8912	827183	NID-100002	5550102
Carol Wang	6617	372912	NID-100003	5550103

ACCOUNT INSTANCE

Account #	Account type	Interest rate	Min. balance	Balance
917322	Savings	4.0%	5000	7812
372912	Current	0.0%	0	291820
827183	Term Deposit	6.75%	10000	100000



Physical Data Independence

Modify the **physical schema**
without changing the **logical schema**.

Applications depend on the logical schema.

Like an interface and its implementation, the levels need well-defined boundaries so internal changes do not unnecessarily affect users.



Data Models and Database Languages



Data Models

Tools for describing **data**, **relationships**, **semantics** and **constraints**.

Relational model

The principal focus of this course.

Entity-relationship model

Used mainly for database design.

Object-based models

Object-oriented and object-relational.

Earlier models

Network and hierarchical models.

Vector model

High-dimensional embeddings support similarity search.

Semi-structured, unstructured, and binary data use suitable representations such as XML documents and BLOBs.



Relational Model

Data are represented in tables. The instructor relation is one example.

ID	name	dept_name	salary	Columns Attributes
22222	Einstein	Physics	95000	
12121	Wu	Finance	90000	
32343	El Said	History	60000	
45565	Katz	Comp. Sci.	75000	
98345	Kim	Elec. Eng.	80000	
76766	Crick	Biology	72000	
10101	Srinivasan	Comp. Sci.	65000	
58583	Califieri	History	62000	Rows Records
83821	Brandt	Comp. Sci.	92000	
15151	Mozart	Music	40000	
33456	Gold	Physics	87000	
76543	Singh	Finance	80000	



A Sample Relational Database

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

department

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000



Data Definition Language (DDL)

A notation for defining the database schema.

```
CREATE TABLE instructor (  
    ID char(5),  
    name varchar(20),  
    dept_name varchar(20),  
    salary numeric(8,2)  
);
```

The DDL compiler records table definitions in the **data dictionary**.

The dictionary stores **metadata**: schema, integrity constraints and authorization information.

Examples of metadata: a primary key identifying instructors; rules defining who can access what.



Data Manipulation Language (DML)

A language for accessing and manipulating the data organized by a data model.

Query languages provide the retrieval part of DML.

Pure languages

Relational algebra
Tuple relational calculus
Domain relational calculus

Practical languages

SQL is the most widely used language in commercial relational systems.



SQL in an Application

- SQL is widely used to define, query and manipulate databases.
- The core relational query language is not a general-purpose programming language; modern extensions add recursion and procedural features.
- Applications often combine SQL with a **host language**, such as C, C++, Java or Python.
- Database access can use embedded SQL or an **API**, such as ODBC or JDBC.



Relational Database Design



Database Design

Designing the general structure of a database.

Logical design

Choose a good collection of relation schemas.

Business decision

What attributes should we record?

Computer science decision

How should attributes be distributed among relations?

Physical design

Choose the physical layout of the database.



Database Design: Is This a Good Relation?

Is there any problem with this table?

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Database Design: A Combined Relation

Is there any problem with this relation?

Redundancy → update anomalies

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Database Design: Separate Relations

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

department

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

Crick → Biology

Department budget: 90000



Database Engine



Database Engine

Storage manager

Query processing

Transaction
manager



Storage Management

The **storage manager** interfaces between low-level stored data and the application programs and queries submitted to the system.

Responsibilities

- Interact with the operating system's file manager
- Store, retrieve, and update data efficiently

Issues

- Storage access
- File organization
- Indexing and hashing

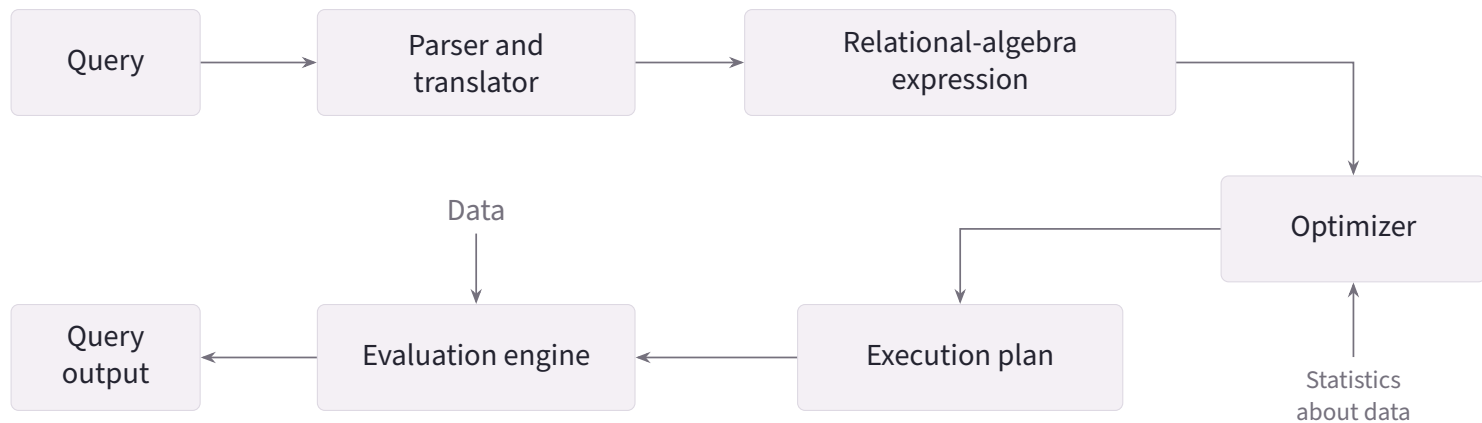


Query Processing

Parsing and translation

Optimization

Evaluation





Transaction Management

What if the system **FAILS**? What if users update the same data **CONCURRENTLY**?

A **transaction** is a collection of operations that performs **one logical function** in a database application.

Transaction management

Preserve a **consistent** database state despite system failures and transaction failures.

Concurrency control

Control interactions among concurrent transactions to preserve consistency.



Lecture Summary

01

Data abstraction separates views, logical structure, and physical storage; **physical data independence** protects this separation

02

Schemas define structure, **instances** hold current data, and the **relational model** represents data as tables

03

DDL defines schemas; **DML** and SQL access and modify data directly or through applications

04

Good design reduces redundancy; the **DBMS engine** manages storage, query plans, and transactions